



Università degli Studi Dell'Insubria
Dipartimento di Scienze Teoriche e Applicate

Laboratorio Interdisciplinare B

Manuale Tecnico: TheKnife

a.a 2025/2026

Data: 30/08/2026

Autori: Luca Cavagnari, Matteo Landini, Francesco Semenzato

Ruoli

Nome	Cognome	Matricola	Ruolo
Luca	Cavagnari	761291	Project Manager / System Architect / Backend
Francesco	Semenzato	760120	System Architect /Backend
Matteo	Landini	753593	Frontend



1. Installazione

1.1 Requisiti di sistema

Componente	Versione richiesta	Note
JDK	17 o superiore	Sufficiente per tutti e tre i moduli (<code>common-api</code> , <code>app-server</code> , <code>app-client</code>).
Maven	3.9.9 o superiore	Build multi-modulo Reactor.
Docker / Docker Compose	versione recente	Necessario per avviare PostgreSQL in container.
PostgreSQL	18	Fornito come immagine Docker (<code>postgres:18</code>) tramite <code>docker-compose.yaml</code> .

Il modulo `app-client/pom.xml` dichiara come proprietà `<maven.compiler.source>25</maven.compiler.source>` e `<maven.compiler.target>25</maven.compiler.target>`, ma il POM padre configura esplicitamente il `maven-compiler-plugin` con `<source>17</source>` e `<target>17</target>` nel blocco `<configuration>` del plugin: questa configurazione ha precedenza sulle proprietà `maven.compiler.*` dei moduli figli. Di conseguenza, JDK 17 è sufficiente per compilare ed eseguire l'intero progetto; la proprietà a 25 in `app-client/pom.xml` resta un residuo non operante, da allineare in una revisione futura del build.

1.2 Setup ambiente

1. Clonare il repository e posizionarsi nella cartella radice (contenente il `pom.xml` del reactor).
2. Avviare il database PostgreSQL tramite Docker Compose:

```
docker compose up -d postgres
```

Il servizio `postgres` espone la porta 5432 con database `mydb`, utente `testuser` e password `test1234` (credenziali di sviluppo, si veda la sezione 5).

3. Attendere che il container risulti "healthy" (l'healthcheck esegue `pg_isready -U testuser -d mydb` ogni 5 secondi).

1.3 Installazione del programma

Dalla cartella radice del progetto:

```
mvn package
```

Il comando produce, tramite `maven-shade-plugin`, i due file JAR eseguibili:

- `app-server/target/theknifeserver-1.0-SNAPSHOT.jar`
- `app-client/target/theknifeclient-1.0-SNAPSHOT.jar`



Si usa `-Dmaven.test.skip=true` anziché il solo `-DskipTests` perché evitare di eseguire inutilmente la suite di test : `-DskipTests` compilerebbe comunque i test, facendo fallire il `package`.

Il `maven-shade-plugin` rilocalizza i package `it.uninsubria.laboratoriob.api` (→ `it.uninsubria.laboratoriob.libs.theknifeapi`) e le classi di `libphonenumber` all'interno di entrambi i file JAR, per evitare conflitti di classpath quando client e server sono distribuiti insieme.

N.B: Al primo avvio è fortemente consigliato di avviare il server con il flag:

- `--update`
- oppure `--update <path/to/michelin_database.csv>` per poter indicizzare i ristoranti del dataset.

Se nessun percorso è passato in input, il programma cercherà automaticamente il file `michelin_my_maps.csv` nella cartella contenente l'eseguibile del modulo server.

2. Progettazione

2.1 Architettura

Il progetto è un *reactor Maven* a tre moduli:

Modulo	Artifact ID	Ruolo
<code>common-api</code>	<code>theknifeapi</code>	Modello di dominio condiviso (entità, enum), interfacce DAO, interfacce di servizio RMI (<code>api/remote/</code>), validatori, canale di heartbeat TCP condiviso. Nessuna dipendenza da altri moduli del progetto.
<code>app-server</code>	<code>theknifeserver</code>	Persistenza su PostgreSQL (HikariCP), import CSV, cache in memoria (repository), implementazioni RMI lato server, server di heartbeat.
<code>app-client</code>	<code>theknifeclient</code>	GUI JavaFX (punto d'ingresso applicativo), cache locale su file JSON, client RMI, client di heartbeat.

common-api (theknifeapi)

- app-server (theknifeserver) – *Dipendenze: HikariCP, driver PostgreSQL, SLF4J*
- app-client (theknifeclient) – *Dipendenze: Jackson, JavaFX 21*

Architettura client/server. Il server è l'unica autorità sui dati persistenti (PostgreSQL); il client mantiene una cache locale su file JSON (directory `data/`) che gli permette di



ridurre le chiamate RMI per dati già noti. La comunicazione

applicativa client server avviene tramite **Java RMI**: quattro interfacce di servizio

in **common-**

api/api/remote/ (**AuthServiceInter**, **RestaurantServiceInter**, **ReviewServiceInter**, **FavouriteServiceInter**), implementate lato server in **app-**

server/.../server/remote/ (ciascuna un **UnicastRemoteObject** registrato sul registro RMI in ascolto sulla porta 1099) e utilizzate lato client

tramite **RmiRepository** (**app-**

client/.../client/utils/RmiRepository.java).

Ruolo del TCP heartbeat. **HeartbeatChannel** (in **common-api/api/utils/**)

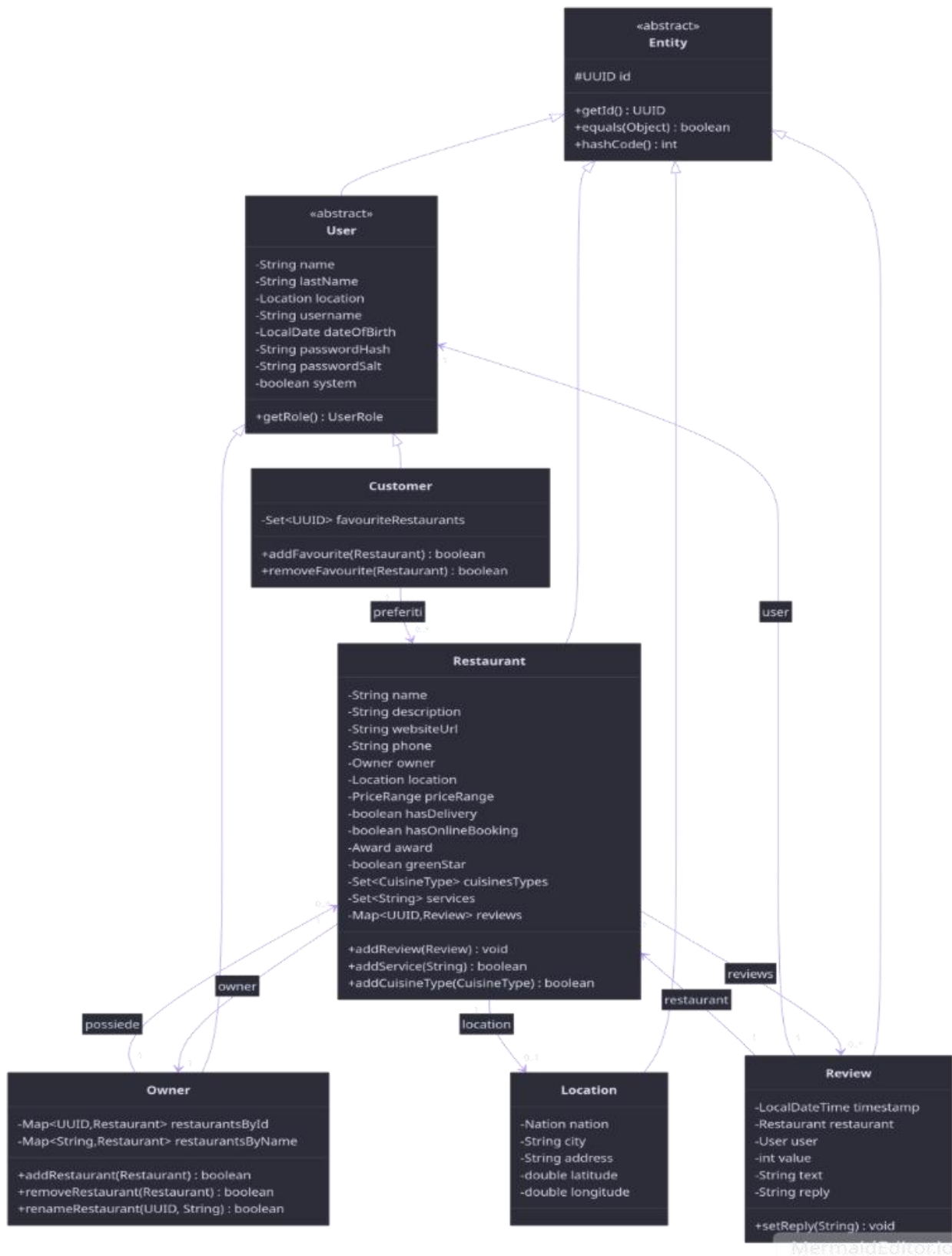
implementa un protocollo PING/PONG bidirezionale su un socket TCP dedicato (porta 5555), distinto dal canale RMI: *due thread daemon* gestiscono rispettivamente l'invio e la ricezione, con un ping ogni 5 minuti (intervallo configurabile) e un timeout di 10 secondi sulla risposta. Il suo unico scopo è il monitoraggio della raggiungibilità della connessione, non lo scambio di dati applicativi.

Ruolo della cache JSON lato client. **ClientDataStore** (facade, sezione 2.6) espone quattro DAO

(**JsonCustomerDAO**, **JsonOwnerDAO**, **JsonRestaurantDAO**, **JsonReviewDAO**) che leggono/scrivono file JSON tramite Jackson **ObjectMapper**, con scrittura atomica (file temporaneo + **Files.move(..., ATOMIC_MOVE)**). Ogni DAO consulta prima la cache locale; se vuota, interroga lo stub RMI corrispondente e popola sia la cache in memoria sia il file su disco; altrimenti legge tutti i dati dalla cache. Le scritture (**save**, **update**, **delete**) seguono uno schema "local-first": la modifica è applicata alla cache/file locale e solo dopo propagata via RMI; se la chiamata RMI fallisce viene sollevata una **ServiceUnavailableException**, ma la modifica locale resta applicata (implicazioni discusse in sezione 5).

2.2 Modello di dominio e schema concettuale

Diagramma delle classi del dominio condiviso (**common-api/api/objects**), ad un livello di astrazione superiore rispetto al codice sorgente:



Enum di supporto: **UserRole** (CLIENT, OWNER), **Award** (6 valori), **PriceRange** (4 fasce, derivate dal numero di simboli \$), **CuisineType** (circa 249 valori) e **Nation** (circa 75)

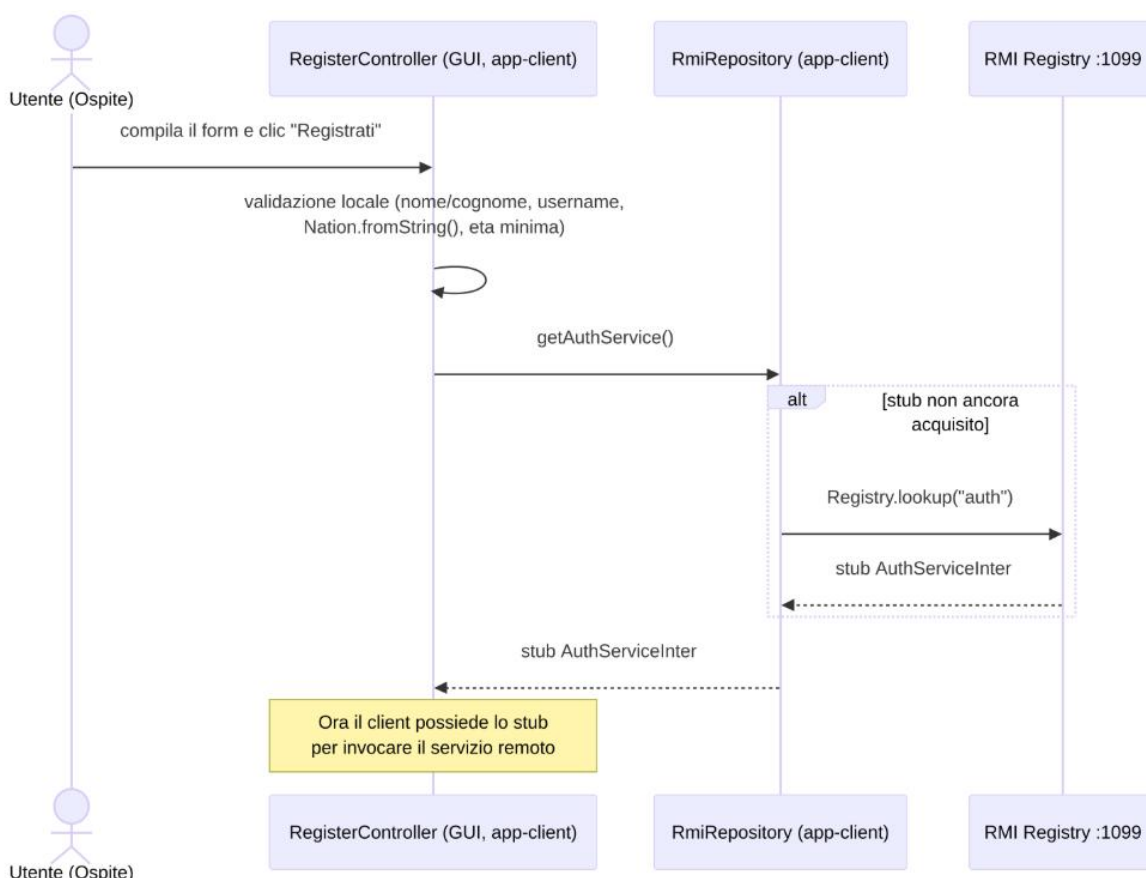
nazioni, con una mappa di alias per il riconoscimento testuale si veda la sezione 5 per un'osservazione sul suo utilizzo).

Dallo schema concettuale allo schema logico. A livello concettuale, **Customer** e **Owner** sono specializzazioni (generalizzazione/ISA) dell'entità **User**, come mostrato nel diagramma sopra. Nella ristrutturazione verso lo schema logico relazionale (sezione 2.4), questa gerarchia viene **"collassata" in un'unica tabella "user"** con un attributo discriminante booleano (**is_owner**), secondo la tecnica standard "collapse hierarchy to single table": si evita così una **JOIN** per ogni accesso ai dati comuni, al costo di alcuni attributi non applicabili a entrambi i sottotipi (nessuno, in questo caso, poiché le informazioni specifiche di **Customer/Owner**, preferiti, ristoranti posseduti, sono rappresentate da tabelle associative separate e non da colonne).

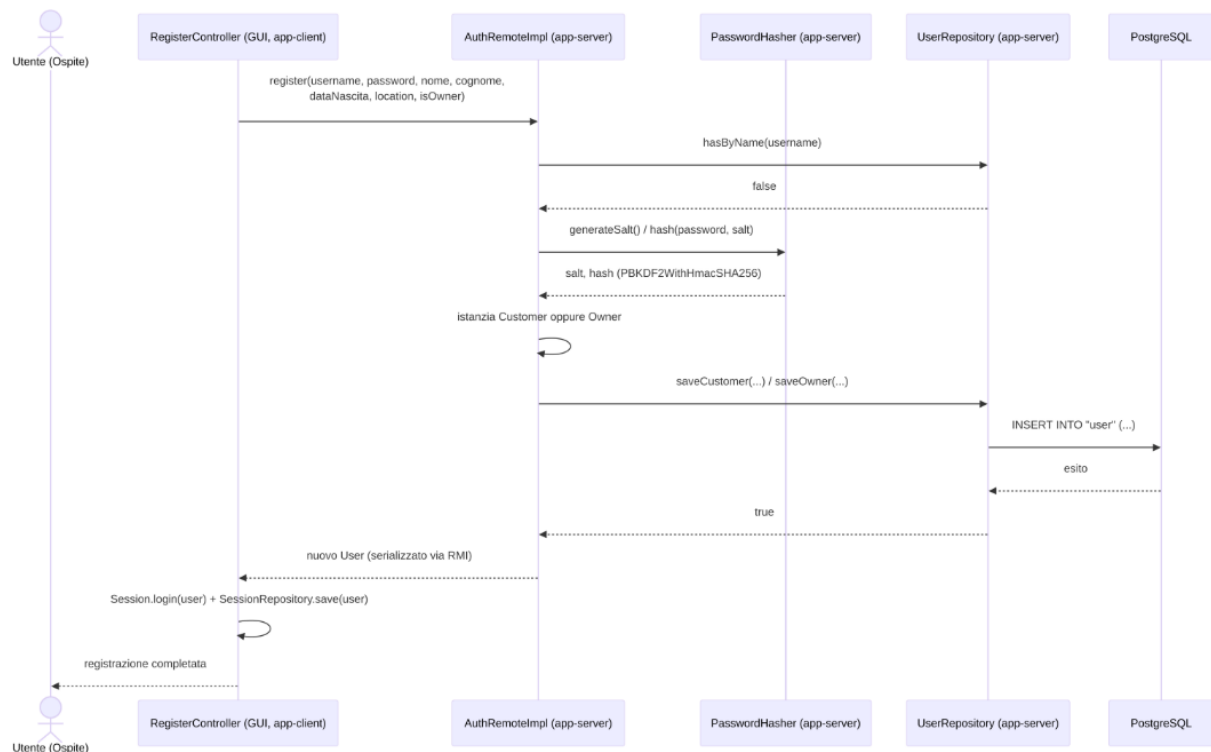
2.3 Interazioni dinamiche

- **Sequence diagram: Registrazione di un nuovo utente via RMI**, ricostruito da **RegisterController**, **RmiRepository**, **AuthServiceInter**, **AuthRemoteImpl**, **UserRepository** e **PasswordHasher**:
- **State diagram: Navigazione della GUI e persistenza della sessione**, ricostruito da **GUIController**, **Navigation** e **Session/SessionRepository**:

2.3.1 Inizializzazione lato client RMI e validazione

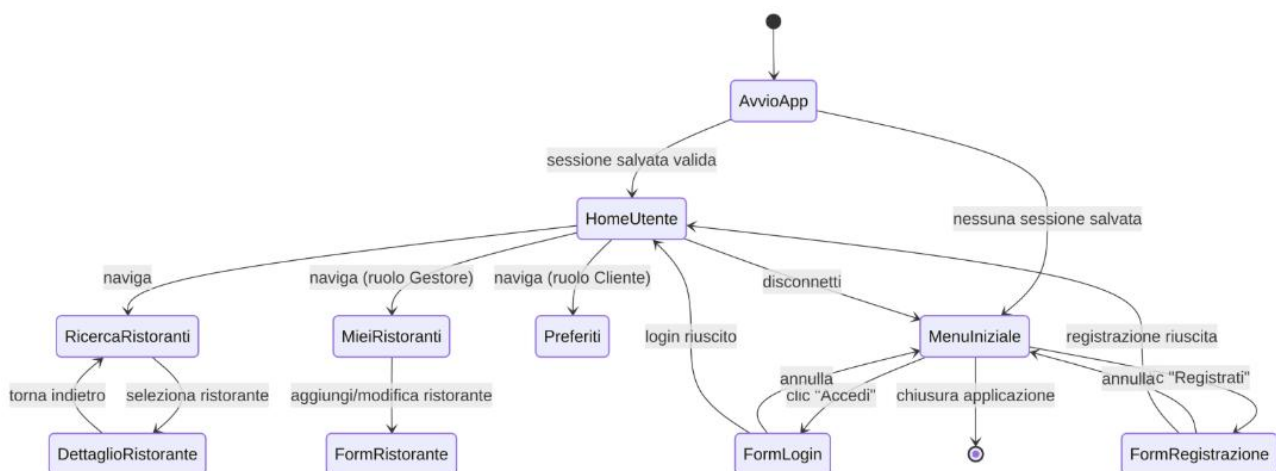


2.3.2: Validazione lato server e persistenza



Il login segue lo stesso schema tramite `AuthServiceInter.login(username, password)`: il server recupera l'utente per username e verifica la password con `PasswordHasher.verify()`, confrontando l'hash calcolato con quello memorizzato.

State diagram: Navigazione della GUI e persistenza della sessione, ricostruito da `GUIController`, `Navigation` e `Session/SessionRepository`:



Il modulo **Navigation** mantiene un backstack indipendente (`Deque<Runnable>`, sezione 2.5) per il pulsante "Indietro" nelle schermate secondarie, distinto dallo stato di autenticazione.

2.4 Base di dati

Analisi dei requisiti ristrutturata (sintesi):

- Un **utente** può essere un **Cliente** o un **Gestore**; a livello concettuale è una generalizzazione (sezione 2.2), ristrutturata in un'unica tabella **"user"** con discriminante **is_owner**.
- Un **ristorante** ha esattamente un proprietario (**restaurant.owner_id NOT NULL REFERENCES "user"(id)**), relazione concettualmente 1-a-molti (un proprietario può possedere più ristoranti). Nello schema fisico questa relazione è rappresentata **due volte**: dalla FK diretta **owner_id** e, in modo ridondante, dalla tabella associativa **user_restaurants** (strutturalmente M:N); si veda l'osservazione in sezione 5.
- Un **cliente** può **marcare come preferiti più ristoranti** e un ristorante può essere preferito da più clienti: relazione molti-a-molti realizzata da **user_favorites**.
- Un **utente non può recensire lo stesso ristorante più di una volta**: vincolo **UNIQUE (user_id, restaurant_id)** su **review**.
- Un ristorante può offrire più tipi di cucina e più servizi: relazioni molti-a-molti tramite **restaurant_cuisine** e **restaurant_services**.
- Una **location** (coppia latitudine/longitudine) è condivisa da **"user"** e **restaurant** tramite chiave esterna composta; la scelta di usare le



coordinate come chiave primaria di `location`, anziché un UUID surrogato, è discussa come limite in sezione 5.

- `price_range`, `awards`, `cuisine_type`, `services_and_facilities` sono tabelle di lookup popolate dagli enum di `common-api`.

Diagramma Entità-Relazione (schema logico, derivato da `sql/create.sql`):

2.4.1: Utenti e posizione geografica

(Dettagli dell'account utente collegati alla loro posizione fisica)



LOCATION		
double	latitude	PK
double	longitude	PK
varchar	city	
char	country	
varchar	address	

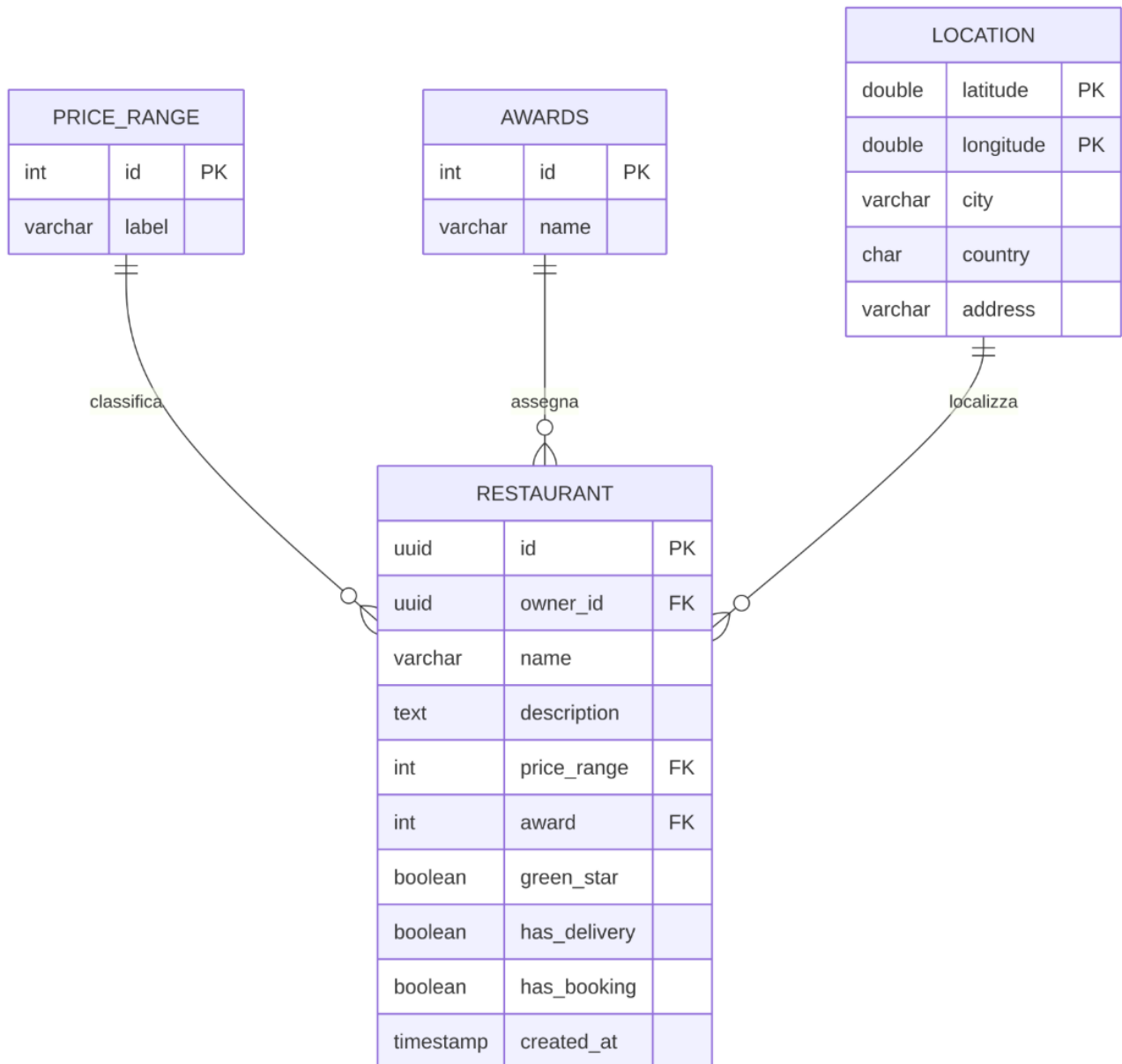


USER			
uuid	id	PK	
varchar	username		UNIQUE
varchar	psw_hash		
varchar	psw_salt		
varchar	first_name		
varchar	last_name		
date	birth_date		
boolean	is_owner		
boolean	is_system		
timestamp	registered_at		



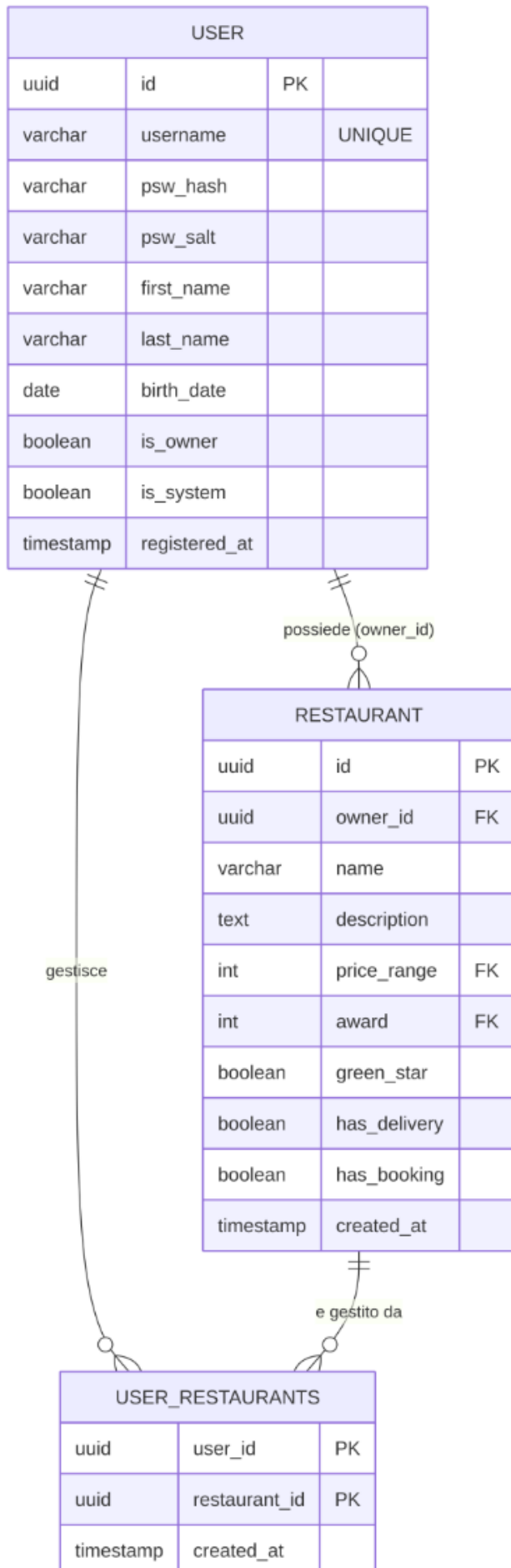
2.4.2: Ristoranti e metriche correlate

(Informazioni sul ristorante, compresa la fascia di prezzo, i riconoscimenti ottenuti e l'ubicazione)



2.4.3: Relazione fra ristoranti e proprietari

(Proprietà diretta e relazione gestionale "molti-a-molti" tramite USER_RESTAURANTS)





2.4.4: Vincoli in linguaggio naturale

(da sql/create.sql):

- Un utente non può nascere nel futuro (`CHECK (birth_date <= CURRENT_DATE)`).
- Il voto di una recensione è compreso tra 1 e 5 (`CHECK (rating BETWEEN 1 AND 5)`).
- Latitudine e longitudine sono vincolate ai rispettivi intervalli geografici validi.
- Un utente non può recensire due volte lo stesso ristorante (`UNIQUE (user_id, restaurant_id)`).
- Una risposta del gestore a una recensione, se presente, non può avere data antecedente alla recensione stessa, e testo/data della risposta devono essere entrambi presenti o entrambi assenti.
- Il nome utente è univoco (`UNIQUE` su `username`).
- Ogni ristorante ha uno ed un solo proprietario (`owner_id NOT NULL`); un proprietario può possedere zero o più ristoranti.
- Le fasce di prezzo hanno un intervallo [`min`, `max`] coerente (`CHECK (max >= min)`).

2.5 Strutture dati e algoritmi

Struttura	Dove	Scopo
<code>ConcurrentHashMap<UUID, T></code> / <code>ConcurrentHashMap<String, T></code>	<code>RestaurantRepository</code> , <code>UserRepository</code> (app-server)	Cache in memoria thread-safe, indicizzata sia per ID sia per nome/username.
<code>AtomicReference<T></code> / <code>AtomicLong</code> / <code>AtomicInteger</code>	<code>RmiRepository</code>	Stato condiviso tra thread per gli stub RMI, i timestamp dell'ultimo tentativo e i contatori di fallimento, senza lock espliciti.
<code>Deque<Runnable></code> (<code>ArrayDeque</code>) come stack	<code>Navigation</code> (client.gui)	Backstack per la navigazione "torna indietro" tra schermate JavaFX.



<code>BlockingQueue<Integer> (ArrayBlockingQueue, capacità 1)</code>	<code>HeartbeatChannel</code>	Sincronizzazione fra il thread che invia il PING e quello che riceve il PONG, con timeout.
<code>Map<UUID, Restaurant> + Map<String, Restaurant></code>	<code>OwnerRepository, RestaurantRepository</code>	Doppio indice (per id e per nome) per lookup senza scansioni lineari.
<code>CompletableFuture<T> (allOf/join)</code>	<code>ServerDataStore#initialise(), CsvParser</code>	Caricamento e import paralleli, con barriera di sincronizzazione.

2.5.1: Scelte algoritmiche principali:

- **Hashing delle password**

(PBKDF2). `PasswordHasher` usa `PBKDF2WithHmacSHA256` con salt casuale a 16 byte, 10.000 iterazioni e chiave a 256 bit, codificata in Base64.

```
// app-server/src/main/java/.../server/utils/PasswordHasher.java
private static String run(char[] password, byte[] saltBytes) {
    PBEKeySpec spec = new PBEKeySpec(password, saltBytes, ITERATIONS, KEY_LENGTH);
    SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
    byte[] hash = skf.generateSecret(spec).getEncoded();
    return Base64.getEncoder().encodeToString(hash);
}
```

- **Parsing del CSV Michelin.** `CsvParser` gestisce formati eterogenei del campo "Location" (città con virgole interne, deduzione della città da codici postali), normalizza la nazione tramite `Nation.valueOf(...)` con fallback su `Nation.fromString(...)`, e corregge automaticamente coordinate lat/lon invertite.



- **Normalizzazione del numero di telefono**, tramite la libreria `libphonenumber` per dedurre il prefisso internazionale dalla nazione della location.
- **Fascia di prezzo da simboli**, `PriceRange.byDollarAmount(int)` mappa il numero di caratteri \$ alle quattro fasce, con `Moderate` come default.
- **Lookup RMI con backoff**. `RmiRepository.lookup(...)` implementa un tentativo con cooldown (massimo un lookup ogni 500 ms) e un limite di fallimenti consecutivi (2), oltre il quale i lookup vengono soppressi fino a un reset esplicito, per evitare di sovraccaricare un registro RMI irraggiungibile con chiamate ripetute.

2.6 Design patterns

- **Pattern Facade**. Le classi `ClientDataStore` (lato client) e `ServerDataStore` (lato server) fungono da *Facade*. Esse incapsulano la complessità dei rispettivi sottosistemi (i DAO JSON e i tre repository), esponendo al resto dell'applicazione un'unica interfaccia semplificata.

```
public class ServerDataStore {  
  
    private final RestaurantRepository restaurants;  
    private final ReviewRepository reviews;  
    private final UserRepository users;  
  
    public ServerDataStore() {  
        this.restaurants = new RestaurantRepository();  
        this.reviews = new ReviewRepository(restaurants);  
        this.users = new UserRepository();  
    }  
  
    public RestaurantRepository restaurants() { return restaurants; }  
  
    public ReviewRepository reviews() { return reviews; }  
  
    public UserRepository users() { return users; }  
}
```

- **DAO (Data Access Object)**. L'interfaccia generica `DAO<T>` (`common-api/api/data/DAO.java`) definisce il contratto CRUD, implementato in due gerarchie indipendenti: lato server tramite JDBC e lato client tramite Jackson/JSON.



```
public interface DAO<T> {  
  
    Optional<T> findById(UUID id);  
  
    List<T> findAll();  
    List<T> findAll(int offset, int limit);  
  
    long count();  
  
    boolean save(T entity);  
    boolean update(T entity);  
    boolean delete(UUID id);  
}
```

- **Repository**

(cache). `RestaurantRepository`, `ReviewRepository`, `UserRepository` incapsulano ciascuno un `DAO` che si interfaccia con connettore JDBC e due `ConcurrentHashMap` di cache; ogni scrittura passa dal DAO e, solo in caso di successo, aggiorna la cache; le letture avvengono dalla cache.

- **Proxy (RMI)**. Gli stub ottenuti da `Registry.lookup(...)` sono proxy locali che inoltrano ogni chiamata al corrispondente `UnicastRemoteObject` sul server. `RmiRepository` aggiunge un livello ulteriore, un proxy pigro con backoff, che acquisisce lo stub reale solo al primo utilizzo e lo ricicla finché valido.

- **Utility class**

(Lombok `@UtilityClass`). `RmiRepository`, `CsvParser`, `Validators` sono



annotate `@UtilityClass`: Lombok genera un costruttore privato e rende statici tutti i membri.

- **Dynamic dispatch / Polimorfismo.** Le entità di dominio `Owner` e `Customer` sono sottotipi della superclasse `User` usata per rappresentare tutti gli utenti, delegando la gestione degli attributi specifici (ristoranti gestiti / ristoranti preferiti, rispettivamente) alle singole classi. Analogamente, sia i DAO, che le repository beneficiano della generalizzazione tramite chiamate alle rispettive superclassi (`UserDAO` e `UserRepository`, rispettivamente) a cui poi effettuano operazioni per assicurarsi la persistenza ed integrità dei loro dati.

3. Esecuzione ed uso

3.1 Setup e lancio del programma

Dopo aver: avviato il database (sezione 1.2) e generato i file JAR(sezione 1.3), l'ordine di avvio è: **prima il server, poi il client.**

3.1.1. Avvio del server

```
$ mvn -B -ntp -DskipTests -Dmaven.test.skip=true -pl app-server run
```

Sequenza di avvio: connessione al pool HikariCP verso PostgreSQL, creazione/verifica dello schema, inserimento delle tabelle costanti (con creazione dell'utente di sistema "System Michelin"), avvio del server di heartbeat TCP sulla porta 5555, caricamento in memoria di utenti e ristoranti, creazione del registro RMI sulla porta 1099 con la registrazione dei quattro stub (`restaurant`, `auth`, `review`, `favourite`).

Per importare il dataset Michelin è disponibile il flag `--update`, opzionalmente seguito dal percorso del CSV (sezione 1.3):

3.1.2. Avvio del client:

```
$ mvn clean compile -am -q && mvn -pl theknifeclient javafx:run
```

`TheKnifeClient` estende `javafx.application.Application`; `main()` invoca `Application.launch(...)`, che carica `gui/GUI.fxml` e apre la finestra principale,



questo è il punto d'ingresso applicativo. Il modulo mantiene anche un'interfaccia testuale (package `client.cli`, classi `GuestMenus`, `CustomerMenus`, `OwnerMenus`, `LoginMenu`), raggiungibile passando il flag `--cli` all'avvio; entrambe le interfacce, GUI e CLI, comunicano con il server tramite le stesse interfacce RMI.2 Uso delle funzionalità

Dal punto di vista funzionale l'applicazione distingue tre ruoli, Ospite, Cliente, Gestore, descritti in dettaglio, con screenshot dell'interfaccia, nel Manuale Utente. Dal punto di vista tecnico, ogni azione dell'utente in GUI corrisponde a una chiamata verso una delle quattro interfacce RMI descritte in sezione 2.1: autenticazione e registrazione (`AuthServiceInter`), ricerca e gestione ristoranti (`RestaurantServiceInter`), scrittura e lettura recensioni (`ReviewServiceInter`), gestione dei preferiti (`FavouriteServiceInter`). Il flusso di registrazione end-to-end è dettagliato come sequence diagram in sezione 2.3.

3.3 Dataset di test

Il file `michelin_my_maps.csv` (radice del repository) contiene un'estrazione del sito Michelin Guide con separatore `;` e colonne: `Name`, `Address`, `Location`, `Price`, `Cuisine`, `Longitude`, `Latitude`, `Phone Number`, `Url`, `WebsiteUrl`, `Award`, `GreenStar`, `FacilitiesAndServices`, `Description`. Viene importato tramite `CsvParser.parseFromDataset(...)`, invocato dal flag `--update` di `TheKnifeServer`. Ogni riga è convertita in un `Restaurant` associato all'utente di sistema "System Michelin" (`is_system = true`, UUID deterministico per garantire idempotenza tra import successivi). I campi `hasDelivery` e `hasOnlineBooking`, assenti nel dataset originale, sono valorizzati casualmente in fase di import (sezione 3).



```
$ mvn -B -ntp -DskipTests -Dmaven.test.skip=true -pl app-server run --update
```

4. JavaDoc

La JavaDoc descrive il contratto di ogni classe, interfaccia e metodo pubblico del progetto, seguendo le convenzioni standard del linguaggio (tag `@param`, `@return`, `@throws`, `@author`).

Generazione. La documentazione HTML si genera con lo strumento `javadoc` del JDK, ad esempio per un singolo modulo:

```
javadoc -d target/javadoc -sourcepath common-api/src/main/java -subpackages  
it.uninsubria.laboratoriob.api
```

oppure, in modo integrato con Maven, tramite il goal del plugin ufficiale:



mvn javadoc:javadoc

Consultazione. La documentazione generata e pubblicata all'indirizzo [javadoc-docsvault.vercel.app/projects/theknifeproject_b](https://docsvault.vercel.app/projects/theknifeproject_b)

5. Limiti della soluzione sviluppata

1. **Incoerenza tra placeholder e validazione del campo "Nazione".** I moduli di registrazione e di inserimento ristorante mostrano il placeholder "Nazione (es. Italy)", ma la mappa di alias di `Nation.java` riconosce solo "it" e "italia", non "italy": digitare il suggerimento del placeholder produce l'errore "Nazione non riconosciuta".
2. **Localizzazione incompleta e incoerente nella GUI.** La schermata di dettaglio ristorante (`RestaurantDetails.fxml`) mescola etichette in inglese (DESCRIPTION, WEBSITE, PHONE NUMBER, PRICE RANGE, BEST REVIEW, WORST REVIEW, il pulsante "← Back") con il resto dell'interfaccia in italiano. Non esiste un meccanismo di internazionalizzazione: le stringhe sono cablate nei file FXML e nei controller.
3. **Validazione del testo delle recensioni non sempre prevedibile.** In un test manuale, un testo di recensione contenente il carattere ! è stato respinto con l'errore "Il valore inserito contiene caratteri non validi o ha un numero di caratteri non consentito (4-200)", mentre lo stesso testo privo di punteggiatura è stato accettato: l'espressione di validazione andrebbe rivista per ammettere la punteggiatura comune.
4. **Schema di consistenza "local-first" tra client e server.** Le scritture sui DAO JSON del client applicano la modifica alla cache/file locale prima di propagarla via RMI; se la chiamata RMI fallisce, la modifica locale non viene annullata, senza un meccanismo di riconciliazione automatica.
5. **Ridondanza nella rappresentazione della proprietà di un ristorante.** La relazione Gestore-Ristorante, concettualmente 1-a-molti, è rappresentata sia dalla FK diretta `restaurant.owner_id` sia dalla tabella associativa M:N `user_restaurants` (sezione 2.4); le due rappresentazioni andrebbero unificate.
6. **Import CSV: valori generati casualmente per campi assenti nel dataset.** `hasDelivery` e `hasOnlineBooking` sono assegnati tramite `SecureRandom.nextBoolean()`, poiché il dataset Michelin originale non contiene queste informazioni: i valori importati su questi due campi non hanno significato informativo reale.
7. **Credenziali del database in chiaro.** Le credenziali PostgreSQL (`testuser / test1234`) sono definite in chiaro in `docker-compose.yaml` e in `Database.java`, accettabile per un progetto didattico, non per un deployment di produzione.

6. Sitografia e Bibliografia



- Oracle: *The javadoc Tool*, Online: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- Oracle: *Java RMI*, package `java.rmi` (Java SE 17 API), Online: <https://docs.oracle.com/en/java/javase/17/docs/api/java.rmi/module-summary.html>
- Project Lombok, Online: <https://projectlombok.org/>
- FasterXML Jackson, Online: <https://github.com/FasterXML/jackson>
- HikariCP, Online: <https://github.com/brettwooldridge/HikariCP>
- Google: libphonenumber, Online: <https://github.com/google/libphonenumber>
- PostgreSQL Global Development Group, Online: <https://www.postgresql.org/>
- Mermaid: libreria di diagrammi testuali usata per i diagrammi di questo documento, Online: <https://mermaid.js.org/>
- Javadoc del progetto, Online: https://javacode-docsvault.vercel.app/projects/theknifeproject_b/index.html
- Università degli Studi dell'Insubria, Dipartimento di Scienze Teoriche e Applicate, "Laboratorio Interdisciplinare B, Documentazione di Progetto", G. Meroni.